

CS358 Final Project Instructions

Part B

Overview

In this part of the project, you will convert a provided sequential image edge detection program into a parallel implementation using Java's `ExecutorService` API.

The sequential implementation applies the Sobel operator across every interior pixel of an image, computing gradient magnitudes in horizontal and vertical directions. Your task is to redesign the work distribution so that multiple threads execute concurrently, reducing total wall-clock runtime on multi-core hardware.

Background

3.1 The Sobel Edge Detection Algorithm

The Sobel operator estimates the gradient of image intensity at each pixel by convolving the grayscale image with two 3×3 kernels:

Gx (horizontal)	Gy (vertical)
-1 0 +1	-1 -2 -1
-2 0 +2	0 0 0
-1 0 +1	+1 +2 +1

For each interior pixel (x, y) , the gradient magnitude is computed as: $\text{magnitude} = \min(255, \sqrt{G_x^2 + G_y^2})$. The result is packed into a grayscale RGB integer and written to the output image. Because each output pixel depends only on the 3×3 neighborhood of the input image (which is never modified), all pixels can be computed in parallel with no data races.

3.2 Java `ExecutorService`

`ExecutorService` is the standard Java API for managing a pool of worker threads. Key classes and methods you will use:

- `Executors.newFixedThreadPool(n)` — creates a pool of n worker threads.
- `executor.submit(Runnable task)` — enqueues a unit of work for execution.
- `executor.shutdown()` — signals no more tasks will be submitted.

- `CountDownLatch(count)` — a synchronization aid; the main thread calls `await()` and blocks until count reaches zero via `countDown()` calls from worker threads.

What is in Serial Version (`EdgeDetectionSeq.java`)

You are given **`EdgeDetectionSeq.java`**, a fully working sequential implementation. Study it carefully before writing your parallel version.

Your Task: Create `EdgeDetectionParallel.java`

Task 1 — Set Up Your Project

Unzip the provided project folder, `StarterCodeFolderImgProcessing.zip`. The folder has the following directory structure. Confirm the sequential program runs correctly before making any changes:

```
project/
├── EdgeDetectionSeq.java      (provided)
├── EdgeDetectionParallel.java (you will create this)
└── images/
    ├── input/  <-- place your test image here, one image has been provided
    └── output/ <-- generated output images will appear here
```

Compile and run the sequential version to confirm correct output:

```
javac EdgeDetectionSeq.java
java  EdgeDetectionSeq
```

Or use VS Code to open the project folder and run the java source file.

Task 2 — Implement `EdgeDetectionParallel.java` (55 pts)

Create a new file called **`EdgeDetectionParallel.java`** that parallelizes the edge detection loop using Java's `ExecutorService` framework. Your implementation must satisfy the following requirements:

2.1 Thread Pool Creation

- Use `Executors.newFixedThreadPool(n)` to create your thread pool, where `n` is a configurable variable (not a hard-coded magic number).

- Name the variable `numberThreads` (or similar) and set its default value to 8.

2.2 Work Distribution

- Divide the image processing work across threads at the row level: each submitted task processes one complete row of pixels (all `x` values for a given `y`).
- Submit tasks for all rows `y` from 1 to `height - 2` (inclusive), matching the sequential loop bounds.

2.3 Synchronization

- Use a `CountDownLatch` initialized to `height - 2` to ensure the main thread waits until all row tasks complete before stopping the timer.
- Each row task must call `latch.countDown()` after finishing its row.
- Call `latch.await()` on the main thread before calling `executor.shutdown()`.
- The timing block (`startTime` to `endTime`) must enclose the full parallel computation, including the `latch.await()` call.

2.4 Helper Methods

Copy the `computeEdgePixel` and `rgbToGray` helper methods exactly from the sequential version. Do not modify their logic. These are correct and do not need to change.

2.5 Output

- Print the elapsed time in milliseconds using the same format as the sequential version: `Execution time: X ms`.
- Write the result image to `images/output/image_edges_parallel.jpg`.

Task 3 — Performance Analysis

Run your parallel implementation with the following thread counts and record the execution time for each. Use the same input image for every run. Perform two runs per configuration and report the average time.

Number of Threads	Run 1 (ms)	Run 2 (ms)	Avg Time (ms)	Speedup $T_{\text{sequential}} / T_{\text{parallel}}$
Sequential (1 thread)				
2				
4				
8				
16				

After collecting your data, **answer the following questions** in a header comment section in your Java file:

1. **Scalability:** Does the speedup scale linearly with the number of threads? Why or why not? Reference Amdahl's Law in your answer.
2. **Optimal thread count:** Which thread count produced the best performance on your machine? Explain why increasing threads beyond that point may not continue to improve performance.
3. **Overhead:** Identify at least two sources of parallel overhead specific to this implementation.

Part-B Submission

Within your final project zipped folder, include your completed **EdgeDetectionParallel.java** file (with discussion answers in the header comment) to the Final Project Exam link on BB. Make sure your code compiles and runs without errors before submitting. As good team programming practice, include the names of all team members in a comment block on top of your code file. Include the threshold table for discussion during your in-class presentation. The presentation file should also be included in the zipped folder.